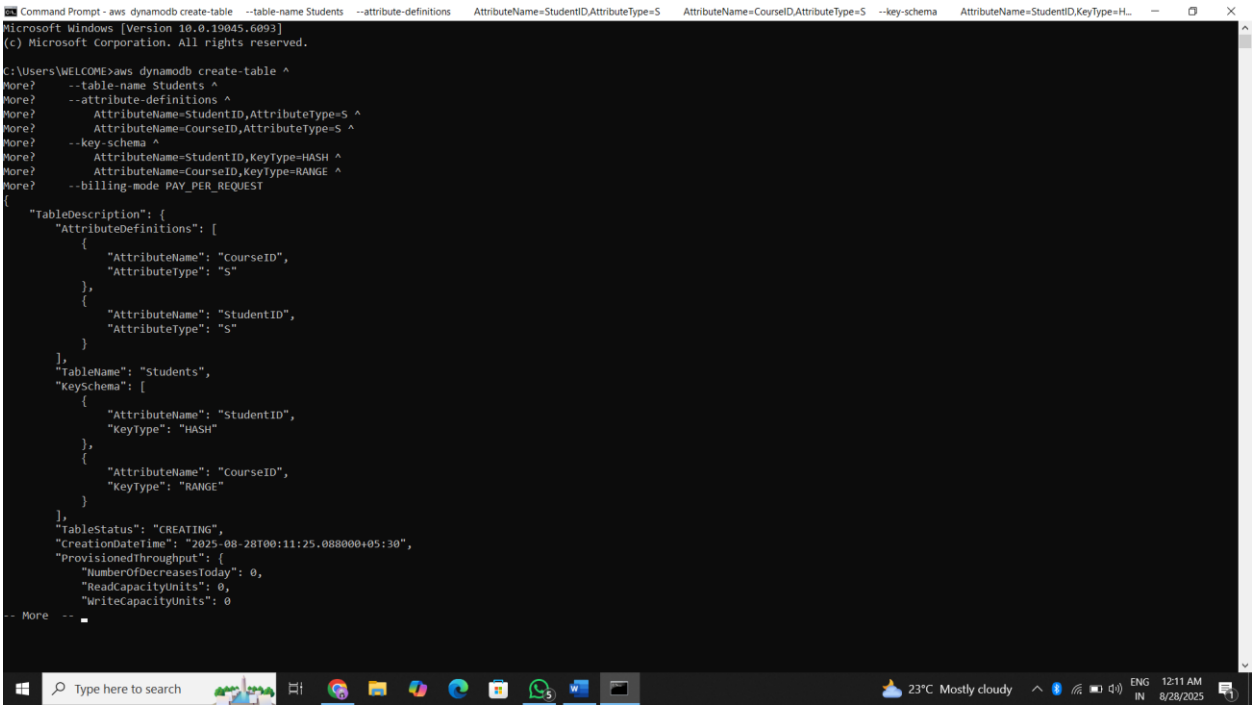
Let’s Create Table Students

We’ll use a composite key:

- StudentID → Partition Key (String)
- CourseID → Sort Key (String)

Code for it :

```
aws dynamodb create-table ^
  --table-name Students ^
  --attribute-definitions ^
    AttributeName=StudentID,AttributeType=S ^
    AttributeName=CourseID,AttributeType=S ^
  --key-schema ^
    AttributeName=StudentID,KeyType=HASH ^
    AttributeName=CourseID,KeyType=RANGE ^
  --billing-mode PAY_PER_REQUEST
```



2. Insert Sample Items

**aws dynamodb put-item ^

  --table-name Students ^

  --item

"{\"StudentID\":{\"S\":\"S001\"},\"CourseID\":{\"S\":\"CSE101\"},\"Name\":{\"S\":\"Amit\"},\"Grade\":{\"S\":\"A\"}}"

**aws dynamodb put-item ^

  --table-name Students ^

  --item

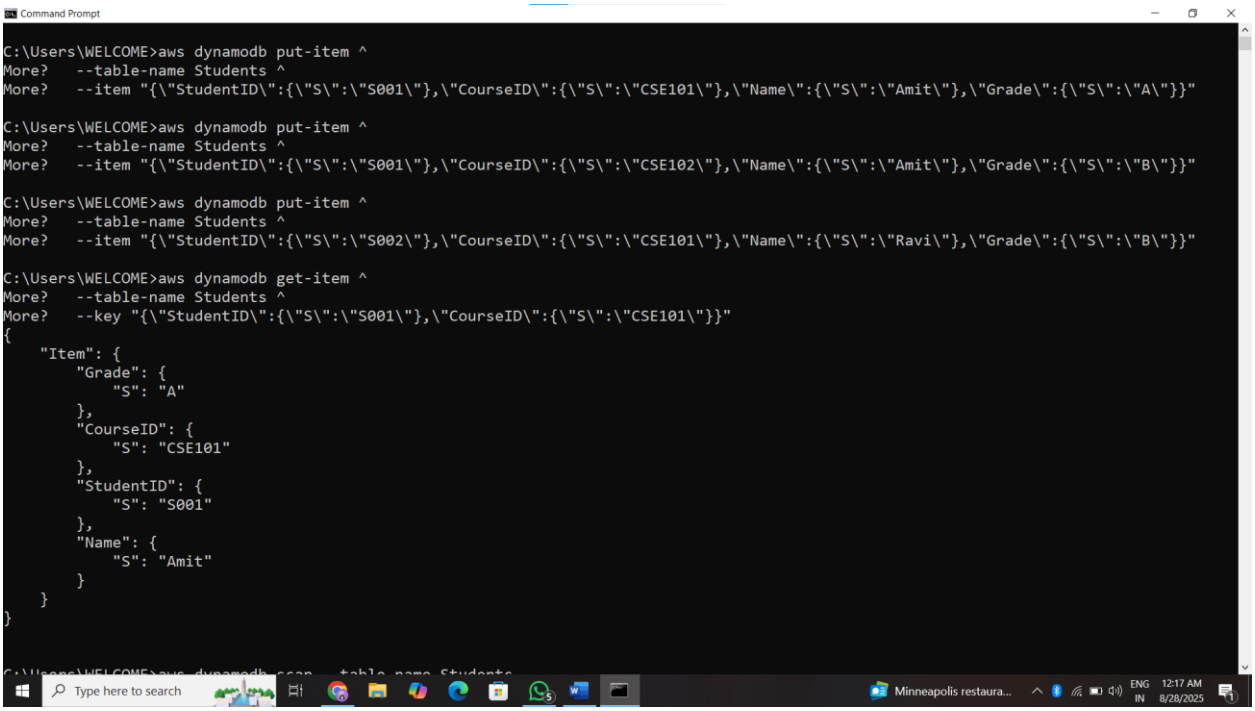
"{\"StudentID\":{\"S\":\"S001\"},\"CourseID\":{\"S\":\"CSE102\"},\"Name\":{\"S\":\"Amit\"},\"Grade\":{\"S\":\"B\"}}"

**aws dynamodb put-item ^

  --table-name Students ^

  --item

"{\"StudentID\":{\"S\":\"S002\"},\"CourseID\":{\"S\":\"CSE101\"},\"Name\":{\"S\":\"Ravi\"},\"Grade\":{\"S\":\"B\"}}"



Then let we get the values of all items.

aws dynamodb scan --table-name Students

```
C:\Users\WELCOME>aws dynamodb scan --table-name Students
{
    "Items": [
        {
            "Grade": {
                "S": "A"
            },
            "CourseID": {
                "S": "CSE101"
            },
            "StudentID": {
                "S": "S001"
            },
            "Name": {
                "S": "Amit"
            }
        },
        {
            "Grade": {
                "S": "B"
            },
            "CourseID": {
                "S": "CSE102"
            },
            "StudentID": {
                "S": "S001"
            },
            "Name": {
                "S": "Amit"
            }
        },
        {
```

Trying Query, Update, and Delete Options

After learning put-item, get-item, and scan, the next step is to cover **querying, updating, and deleting**.

a) Query Items

Queries require a **partition key (and optionally a sort key condition)**.

aws dynamodb query ^

--table-name Students ^

--key-condition-expression "StudentID = :id" ^

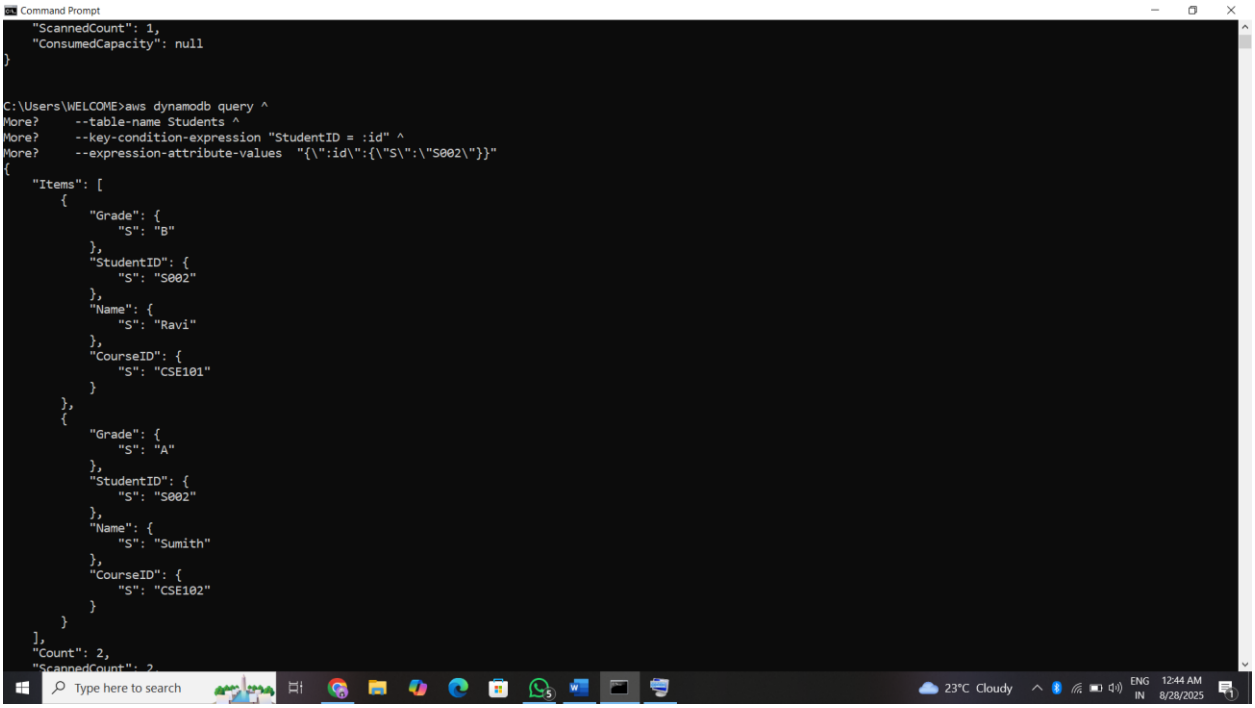
--expression-attribute-values “{\":id\":{\"S\":\"S001\"}}”

```
C:\Users\WELCOME>aws dynamodb query ^
More?   --table-name Students ^
More?   --key-condition-expression "StudentID = :id" ^
More?   --expression-attribute-values "{\":id\":{\"S\":\"S001\"}}"
{
    "Items": [
        {
            "Grade": {
                "S": "A"
            },
            "CourseID": {
                "S": "CSE101"
            },
            "StudentID": {
                "S": "S001"
            },
            "Name": {
                "S": "Amit"
            }
        }
    ],
    "Count": 1,
    "ScannedCount": 1,
    "ConsumedCapacity": null
}

C:\Users\WELCOME>_
```

This will fetch all items where StudentID = "S001".

Doing the same with StudentID = "S002"

```
aws dynamodb query ^
    --table-name Students ^
    --key-condition-expression "StudentID = :id" ^
    --expression-attribute-values  “{\":id\":{\"S\":\"S002\"}}”
```

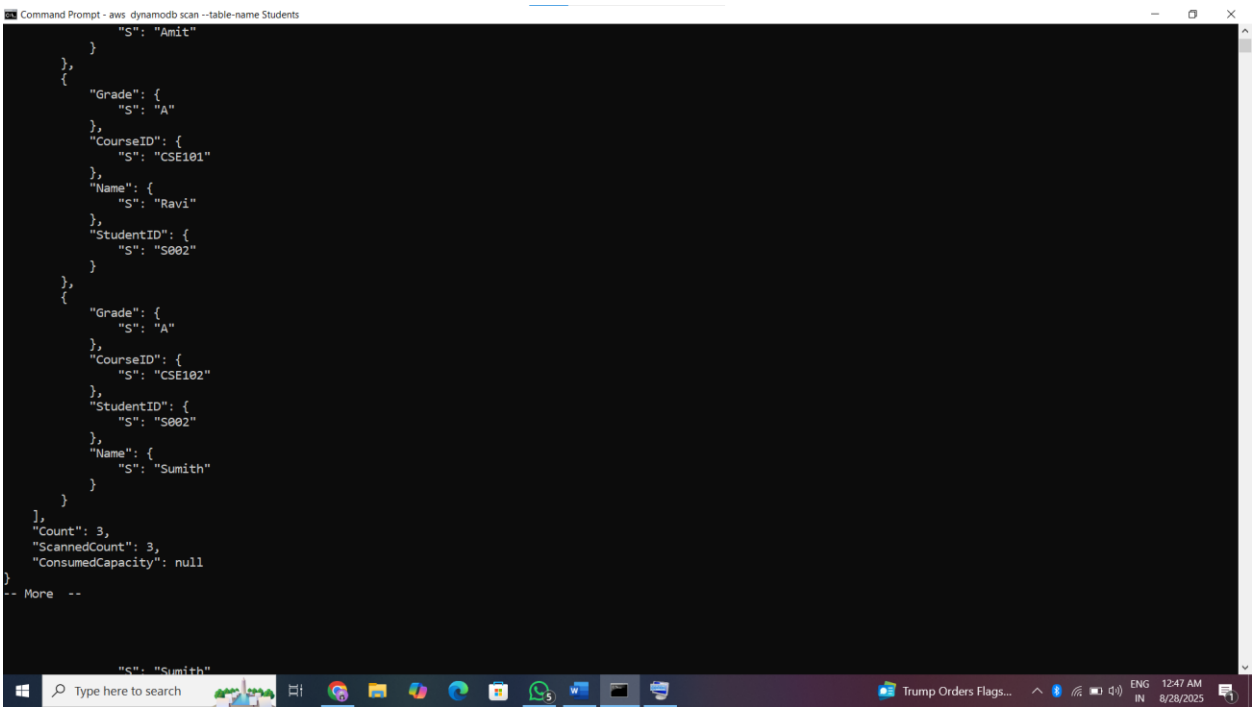


---

b) Update Item

You can update existing attributes or add new ones.

```
aws dynamodb update-item ^
    --table-name Students ^
    --key '{"StudentID":{"S":"S002"}, "CourseID":{"S":"CSE101"}}' ^
    --update-expression "SET Grade = :g" ^
    --expression-attribute-values '{\":g\":{\"S\":\"A\"}}'
```



This updates the grade for student **S002** in course **CSE102**.

---

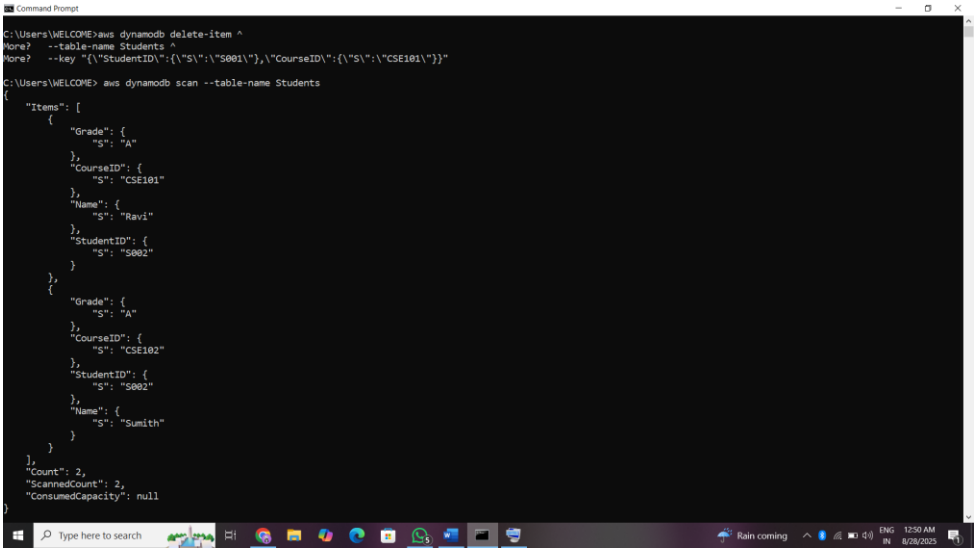
c) Delete Item

Remove an item using its key.

aws dynamodb delete-item ^

  --table-name Students ^

  --key “{"StudentID":{"S":"S001"}, "CourseID":{"S":"CSE101"}}”



---

2. Perform Bulk Insertion from CSV and JSON

a) Bulk Insert from JSON File

Prepare a file students.json with multiple items in **Batch Write Item** format:

```
{
  "Students": [
    {
      "PutRequest": {
        "Item": {
          "StudentID": {"S": "S004"},
          "CourseID": {"S": "CSE104"},
          "Name": {"S": "Suresh"},
          "Grade": {"S": "B"}
        }
      }
    },
    {
      "PutRequest": {
        "Item": {
          "StudentID": {"S": "S005"},
          "CourseID": {"S": "CSE105"},
          "Name": {"S": "Anjali"},
          "Grade": {"S": "A"}
        }
      }
    }
  ]
}
```

students - Notepad

File Edit Format View Help

```
{
  "Students": [
    {
      "PutRequest": {
        "Item": {
          "StudentID": {"S": "S004"},
          "CourseID": {"S": "CSE104"},
          "Name": {"S": "Ravi"},
          "Grade": {"S": "B"}
        }
      }
    },
    {
      "PutRequest": {
        "Item": {
          "StudentID": {"S": "S005"},
          "CourseID": {"S": "CSE105"},
          "Name": {"S": "Anjali"},
          "Grade": {"S": "A"}
        }
      }
    }
  ]
}
```

Ln 25, Col 1 | 130% | Windows (CRLF) | UTF-8

Type here to search | 23°C Cloudy | ENG IN | 12:54 AM 8/28/2025

Run:

aws dynamodb batch-write-item --request-items file://students.json(full directory)

C:\Windows\System32\cmd.exe

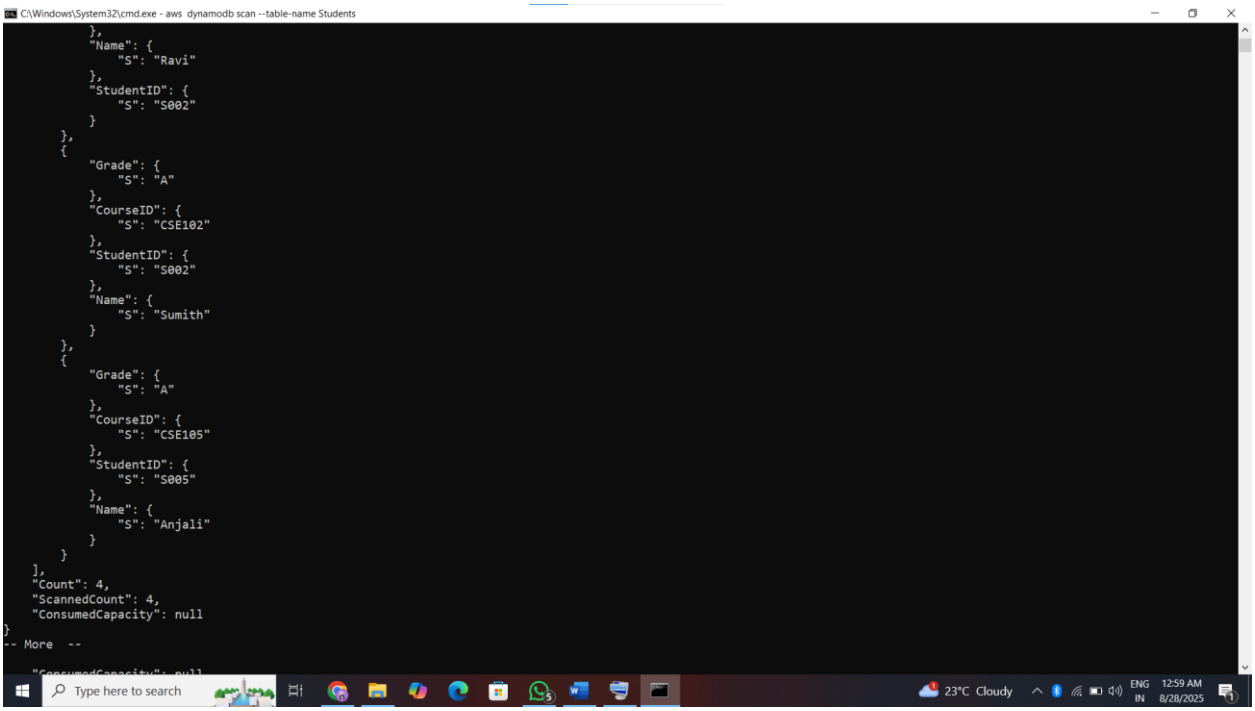
```
Microsoft Windows [Version 10.0.19045.6093]
(c) Microsoft Corporation. All rights reserved.

C:\Users\WELCOME\OneDrive\Desktop>aws dynamodb batch-write-item --request-items file://students.json
{
    "UnprocessedItems": {}
}


C:\Users\WELCOME\OneDrive\Desktop>
```

Type here to search | 23°C Cloudy | ENG IN | 12:59 AM 8/28/2025

That means your **batch-write-item ran successfully** and **all items in students.json were written** to the Students table, since:

```
{
    "UnprocessedItems": {}
}
```

means **no items were skipped or throttled**.

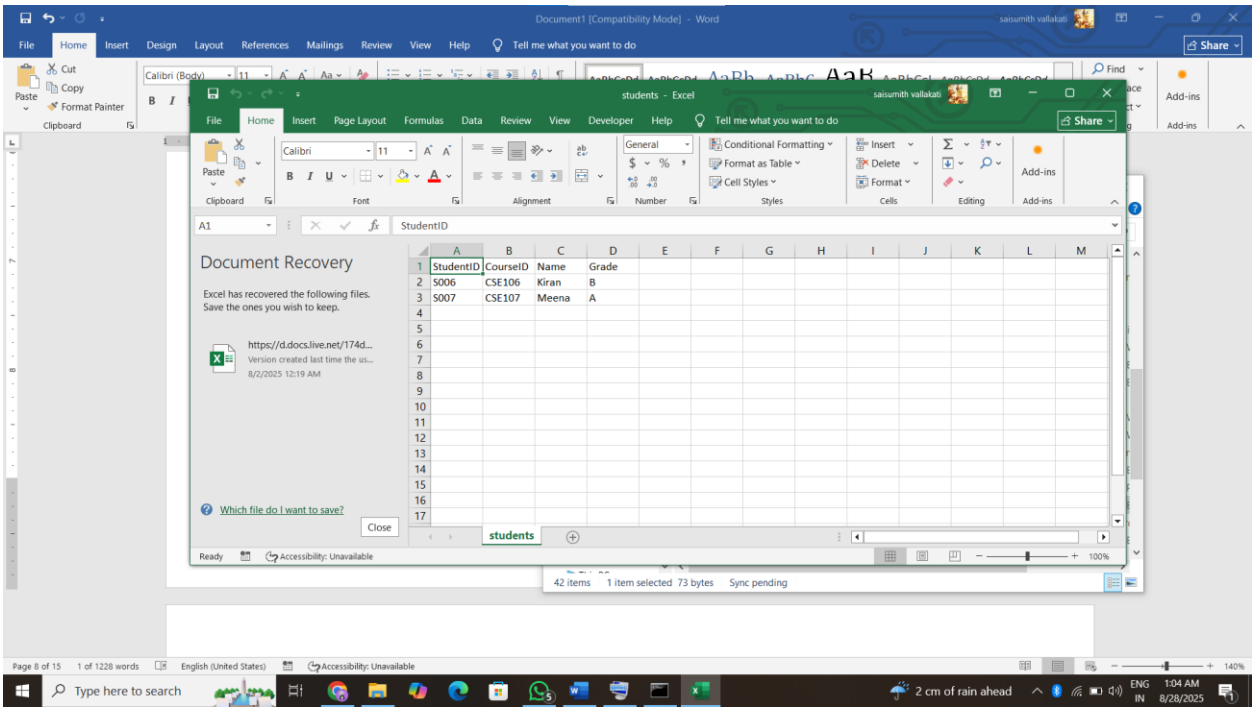


---

b) Bulk Insert from CSV (using Python + Boto3)

If you have a CSV students.csv:

```
StudentID,CourseID,Name,Grade
S006,CSE106,Kiran,B
S007,CSE107,Meena,A
```



Python script to load it:

```
import boto3, csv
dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('Students')
with open('students.csv', newline='') as csvfile:
    reader = csv.DictReader(csvfile)
    with table.batch_writer() as batch:
        for row in reader:
            batch.put_item(Item=row)
print("CSV data inserted successfully!")
```

```python
import boto3, csv
dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('Students')
with open('students.csv', newline='') as csvfile:
    reader = csv.DictReader(csvfile)
    with table.batch_writer() as batch:
        for row in reader:
            batch.put_item(Item=row)
print("CSV data inserted successfully!")
```

```
PS D:\python classworks\DETraining\DE PYTHON> & C:/Users/WELCOME/AppData/Local/Programs/Python/Python313/python.exe "d:/python classworks/DETraining/DE PYTHON/ds/u.py"
CSV data inserted successfully!
PS D:\python classworks\DETraining\DE PYTHON>
```

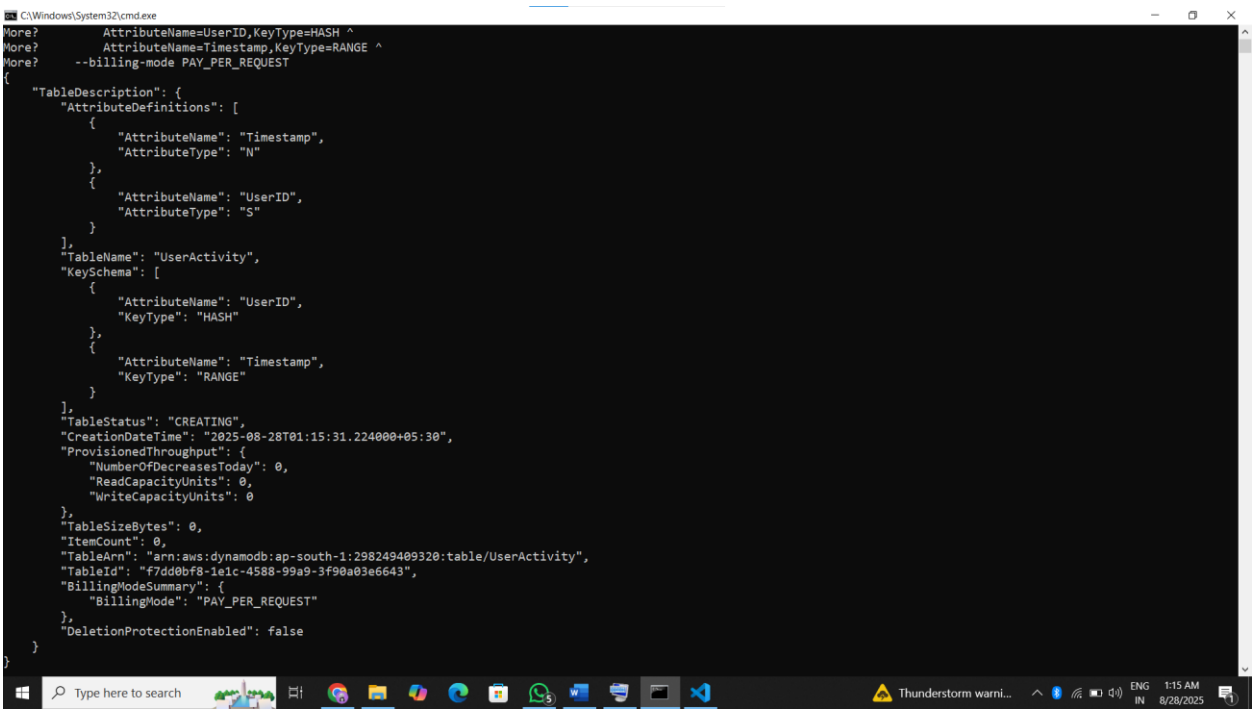
```
C:\Windows\System32\cmd.exe - aws dynamodb scan --table-name Students
            },
            "Name": {
                "S": "Kiran"
            }
        },
        {
            "Grade": {
                "S": "A"
            },
            "CourseID": {
                "S": "CSE105"
            },
            "StudentID": {
                "S": "S005"
            },
            "Name": {
                "S": "Anjali"
            }
        },
        {
            "Grade": {
                "S": "A"
            },
            "CourseID": {
                "S": "CSE107"
            },
            "StudentID": {
                "S": "S007"
            },
            "Name": {
                "S": "Meena"
            }
        }
    ],
    "Count": 6,
    "ScannedCount": 6,
-- More --
```

Lab: Comparing Scan vs Query

1. Create a Table with Composite Key

We'll use UserID as the **partition key** and Timestamp as the **sort key**.

```
aws dynamodb create-table ^
  --table-name UserActivity ^
  --attribute-definitions ^
    AttributeName=UserID,AttributeType=S ^
    AttributeName=Timestamp,AttributeType=N ^
  --key-schema ^
    AttributeName=UserID,KeyType=HASH ^
    AttributeName=Timestamp,KeyType=RANGE ^
  --billing-mode PAY_PER_REQUEST
```



---

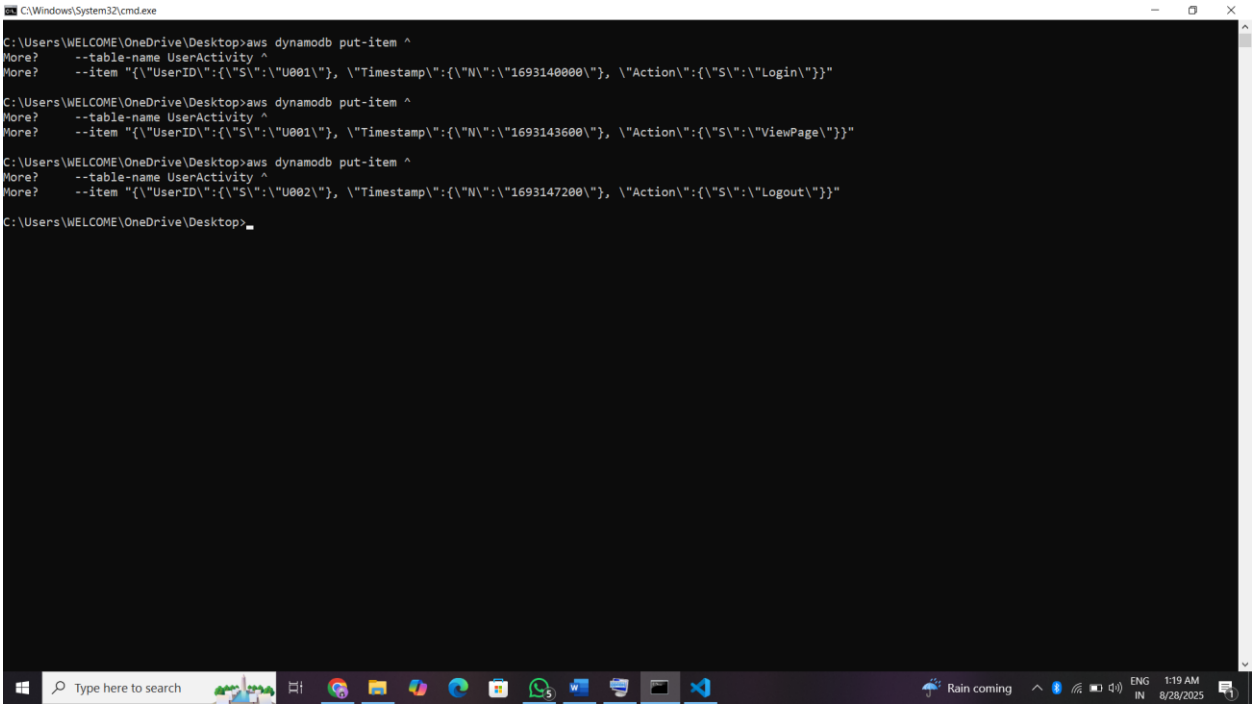
2. Insert Sample Data

Add some records for multiple users.

```
aws dynamodb put-item ^
  --table-name UserActivity ^
  --item '{"UserID":{"S":"U001"}, "Timestamp":{"N":"1693140000"}, "Action":{"S":"Login"}}'
```

```
aws dynamodb put-item ^
   --table-name UserActivity ^
   --item '{"UserID":{"S":"U001"}, "Timestamp":{"N":"1693143600"}, "Action":{"S":"ViewPage"}}'
```

```
aws dynamodb put-item ^
   --table-name UserActivity ^
   --item '{"UserID":{"S":"U002"}, "Timestamp":{"N":"1693147200"}, "Action":{"S":"Logout"}}'
```

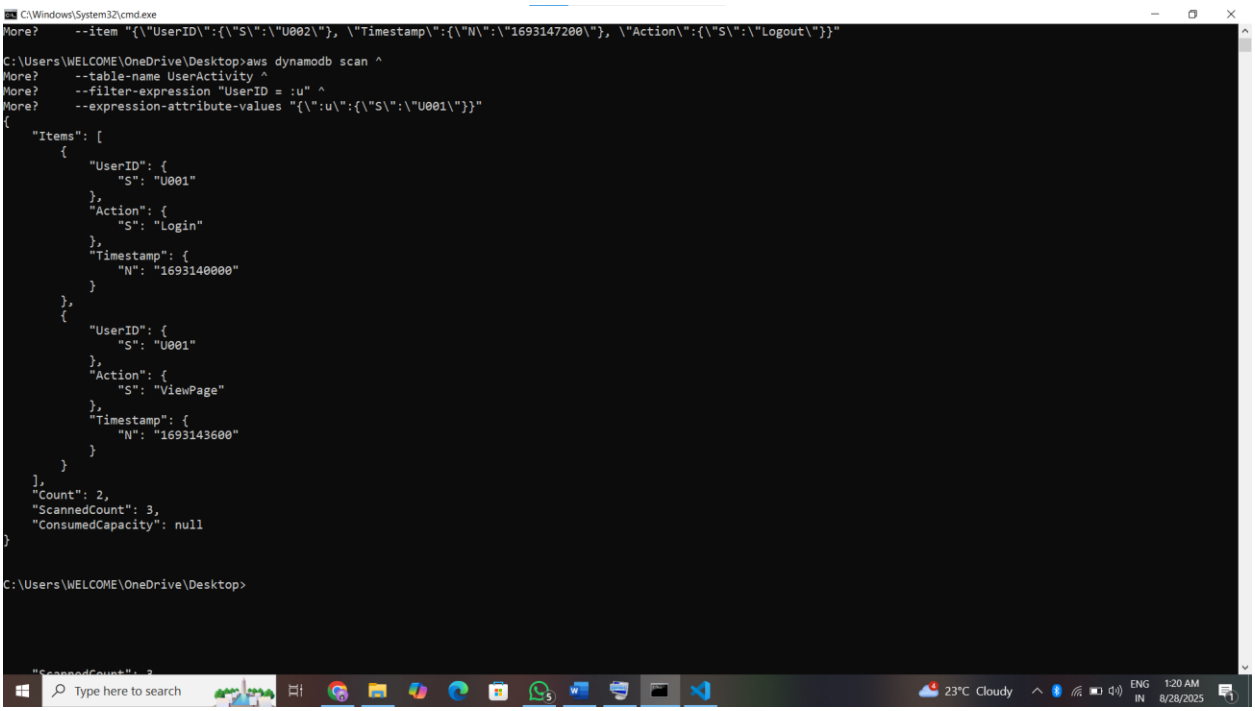


---

3. Use Scan with a Filter

Here, DynamoDB reads **all items** in the table, then applies the filter.

```
aws dynamodb scan ^
   --table-name UserActivity ^
   --filter-expression "UserID = :u" ^
   --expression-attribute-values “{\":u\":{\"S\":\"U001\"}}”
```



Observation:

- DynamoDB scans *every row* and then filters.
- Cost = proportional to **entire table size**, not just result size.
- Very slow for large datasets.

---

4. Use Query

Now use the partition key directly.

```
aws dynamodb query ^
  --table-name UserActivity ^
  --key-condition-expression "UserID = :u" ^
  --expression-attribute-values "{\":u\":{\"S\":\"U001\"}}"
```

```
C:\Windows\System32\cmd.exe
C:\Users\WELCOME\OneDrive\Desktop>aws dynamodb query ^
More?     --table-name UserActivity ^
More?     --key-condition-expression "UserID = :u" ^
More?     --expression-attribute-values "{\":u\":{\"S\":\"U001\"}}"
{
    "Items": [
        {
            "UserID": {
                "S": "U001"
            },
            "Action": {
                "S": "Login"
            },
            "Timestamp": {
                "N": "1693140000"
            }
        },
        {
            "UserID": {
                "S": "U001"
            },
            "Action": {
                "S": "ViewPage"
            },
            "Timestamp": {
                "N": "1693143600"
            }
        }
    ],
    "Count": 2,
    "ScannedCount": 2,
    "ConsumedCapacity": null
}

C:\Users\WELCOME\OneDrive\Desktop>S_
```

Observation:

- DynamoDB jumps straight to partition **U001**.
- Reads only relevant items (fast and cheap).
- Scales linearly with number of items for that user, not table size.

---

5. Learning Goal

- **Scan** = full table scan → expensive & slow → use only for small datasets or one-off admin tasks.
- **Query** = efficient partition search → always preferred for production workloads.
- By design, DynamoDB forces you to think about **access patterns** up front.

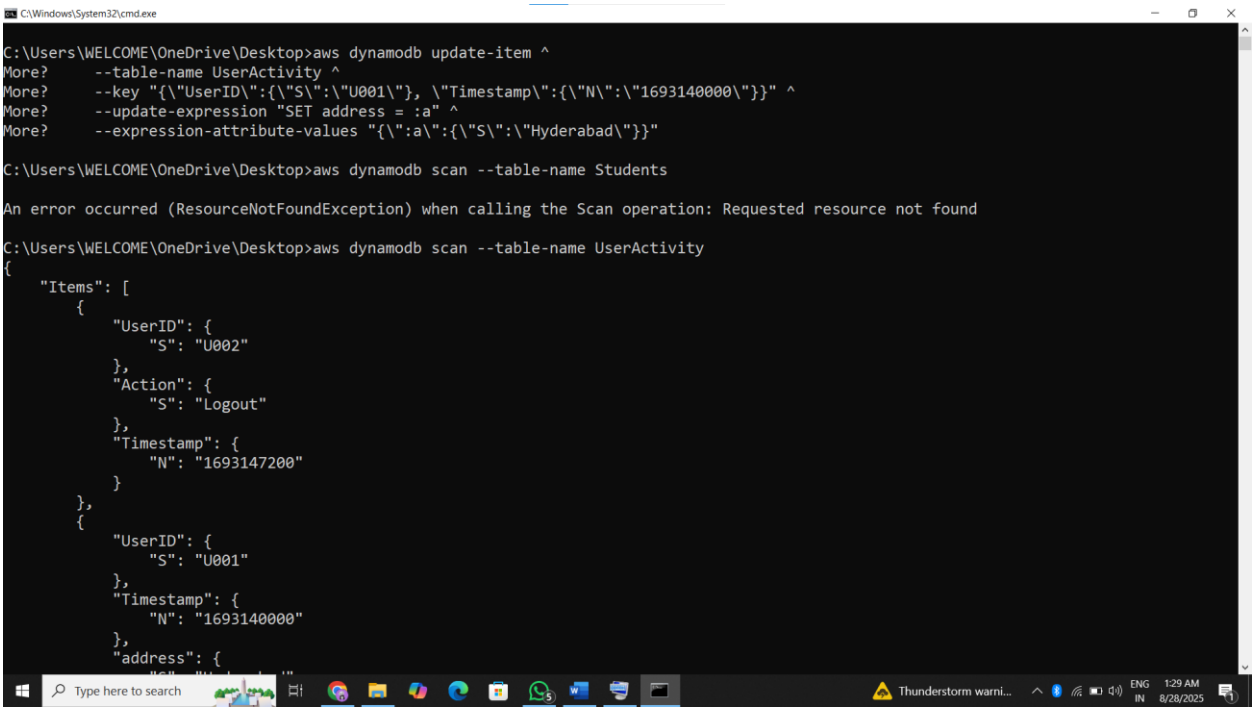
Lab: Update and Delete in DynamoDB

---

1. Update an Existing Item

a) Add a New Attribute (e.g., address)

```
aws dynamodb update-item ^
  --table-name UserActivity ^
  --key "{\"UserID\":{\"S\":\"U001\"}, \"Timestamp\":{\"N\":\"1693140000\"}}" ^
  --update-expression "SET address = :a" ^
  --expression-attribute-values "{\":a\":{\"S\":\"Hyderabad\"}}"
```



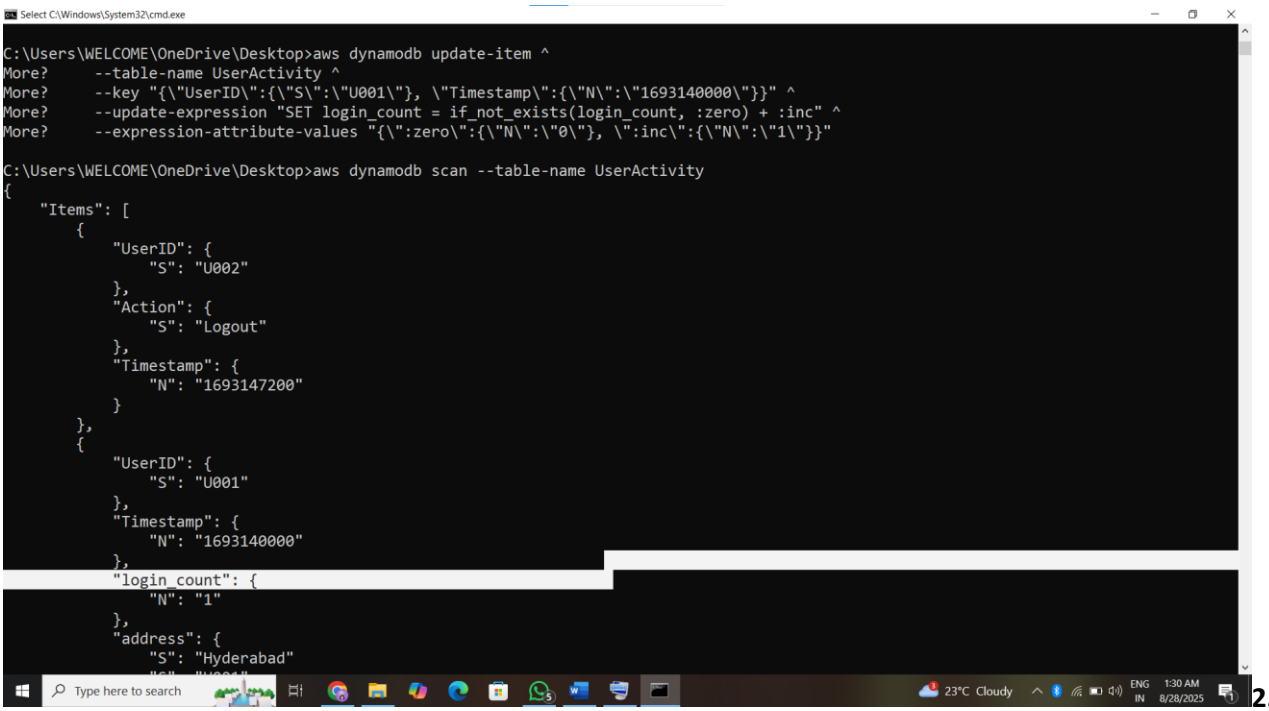
This adds a new attribute address to the item.

---

b) Increment a Number Attribute (login_count)

```
aws dynamodb update-item ^
  --table-name UserActivity ^
  --key "{\"UserID\":{\"S\":\"U001\"}, \"Timestamp\":{\"N\":\"1693140000\"}}" ^
  --update-expression "SET login_count = if_not_exists(login_count, :zero) + :inc" ^
  --expression-attribute-values "{\":zero\":{\"N\":\"0\"}, \":inc\":{\"N\":\"1\"}}"
```

This increases login count by 1 each time (and initializes it to 0 if missing).



2. Conditional Update

Update the item **only if it exists**.

```
aws dynamodb update-item ^
  --table-name UserActivity ^
  --key '{"UserID":{"S":"U001"}, "Timestamp":{"N":"1693140000"}}' ^
  --update-expression "SET address = :a" ^
  --expression-attribute-values '{":a":{"S":"Chennai"}}' ^
  --condition-expression "attribute_exists(UserID)"
```

```
C:\Windows\System32\cmd.exe

C:\Users\WELCOME\OneDrive\Desktop>aws dynamodb update-item ^
More?     --table-name UserActivity ^
More?     --key "{\"UserID\":{\"S\":\"U001\"}, \"Timestamp\":{\"N\":\"1693140000\"}}" ^
More?     --update-expression "SET address = :a" ^
More?     --expression-attribute-values "{\":a\":{\"S\":\"Chennai\"}}" ^
More?     --condition-expression "attribute_exists(UserID)"

C:\Users\WELCOME\OneDrive\Desktop>aws dynamodb scan --table-name UserActivity
{
    "Items": [
        {
            "UserID": {
                "S": "U002"
            },
            "Action": {
                "S": "Logout"
            },
            "Timestamp": {
                "N": "1693147200"
            }
        },
        {
            "UserID": {
                "S": "U001"
            },
            "Timestamp": {
                "N": "1693140000"
            },
            "login_count": {
                "N": "1"
            },
            "address": {
```

This ensures the update only happens if the item is present.

---

3. Delete an Item

Delete using its full key.

```
aws dynamodb delete-item ^
  --table-name UserActivity ^
  --key '{"UserID":{"S":"U001"}, "Timestamp":{"N":"1693140000"}}'
```

```
C:\Windows\System32\cmd.exe
C:\Users\WELCOME\OneDrive\Desktop>aws dynamodb scan --table-name UserActivity
{
    "Items": [
        {
            "UserID": {
                "S": "U002"
            },
            "Action": {
                "S": "Logout"
            },
            "Timestamp": {
                "N": "1693147200"
            }
        },
        {
            "UserID": {
                "S": "U001"
            },
            "Action": {
                "S": "ViewPage"
            },
            "Timestamp": {
                "N": "1693143600"
            }
        }
    ],
    "Count": 2,
    "ScannedCount": 2,
    "ConsumedCapacity": null
}
```

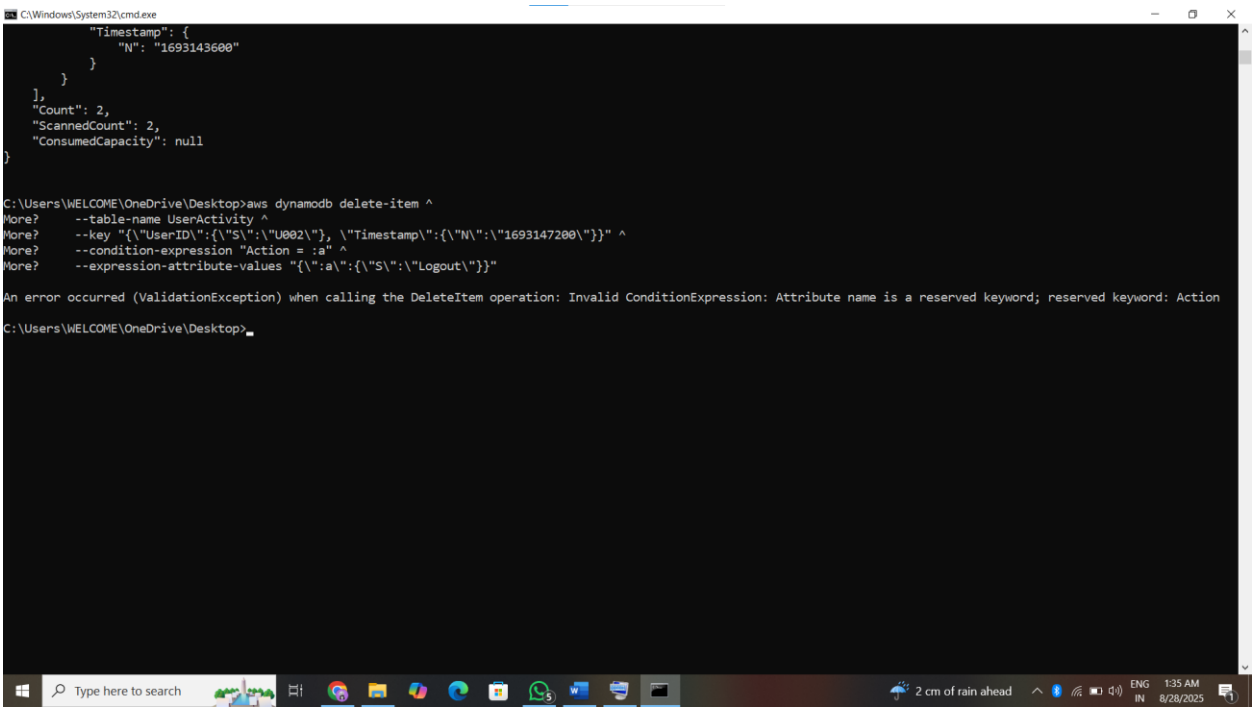
This removes the item completely.

---

4. Conditional Delete

Delete only if a condition is satisfied (e.g., Action = Logout).

```
aws dynamodb delete-item ^
  --table-name UserActivity ^
  --key "{\"UserID\":{\"S\":\"U002\"}, \"Timestamp\":{\"N\":\"1693147200\"}}" ^
  --condition-expression "Action = :a" ^
  --expression-attribute-values "{\":a\":{\"S\":\"Logout\"}}"
```



If the condition fails, DynamoDB **does not delete** and throws a Conditional Check Failed Exception.

---

5. (Optional) Python Script for Update/Delete

```
import boto3
dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('UserActivity')
# Update: increment login_count
table.update_item(
    Key={"UserID": "U001", "Timestamp": 1693140000},
    UpdateExpression="SET login_count = if_not_exists(login_count, :zero) + :inc",
    ExpressionAttributeValues={":zero": 0, ":inc": 1}
)
# Conditional Delete: only if Action = 'Logout'
try:
    table.delete_item(
        Key={"UserID": "U002", "Timestamp": 1693147200},
        ConditionExpression="Action = :a",
        ExpressionAttributeValues={":a": "Logout"}
    )
```

```
    print("Item deleted")
except Exception as e:
    print("Delete failed:", e)
```

Lab: Master the Query Operation in DynamoDB

We'll continue using the **UserActivity** table:

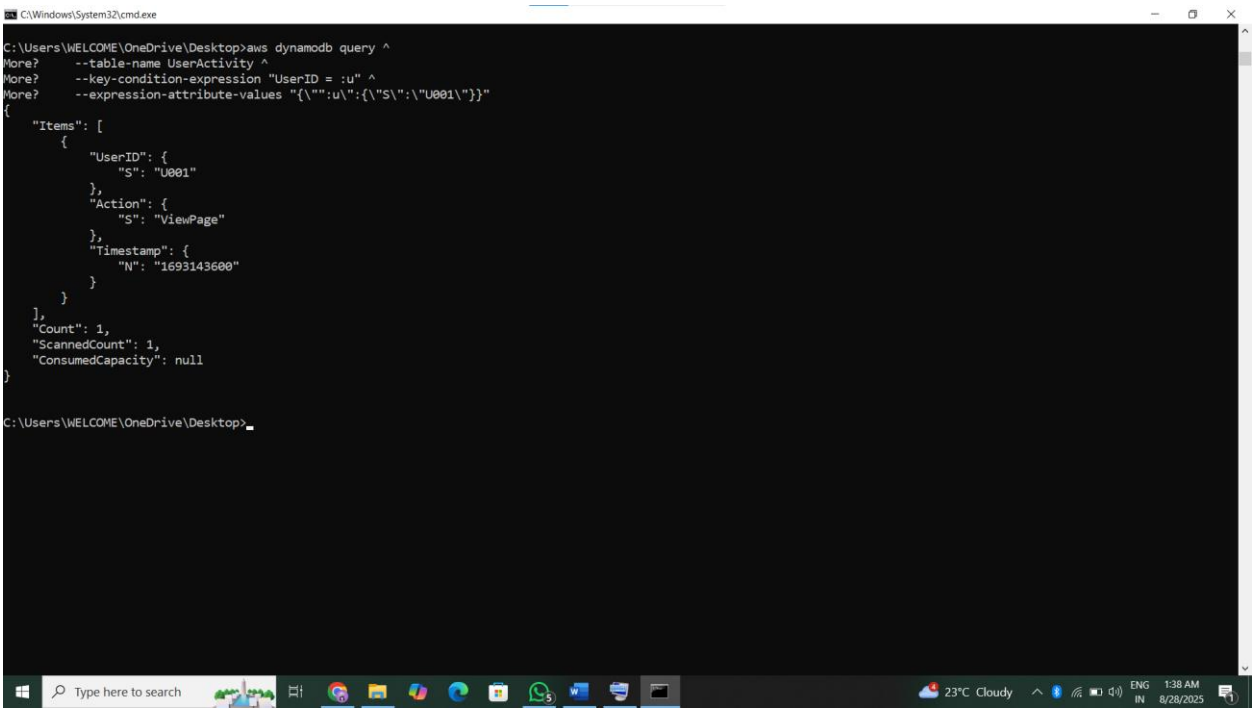
- **Partition Key**: UserID (S)
- **Sort Key**: Timestamp (N)

---

1. Query: All Items for a Single Partition Key

Fetch all activity for user U001:

```
aws dynamodb query ^
    --table-name UserActivity ^
    --key-condition-expression "UserID = :u" ^
    --expression-attribute-values "{\"":u\":{\"S\":\"U001\"}}"
```



Result: All items for partition U001, sorted by Timestamp (ascending by default).

---

2. Query with begins_with on Sort Key

Suppose your Timestamp was stored as a string like 2025-08-27T12:30:00. You can query all items from a given day:

```
aws dynamodb query ^
  --table-name UserActivity ^
  --key-condition-expression "UserID = :u AND #ts BETWEEN :start AND :end" ^
  --expression-attribute-names "{\"#ts\":\"Timestamp\"}" ^
  --expression-attribute-values "{\"":u\":{\"S\":\"U001\"}, \":start\":{\"N\":\"1693140000\"},
\":end\":{\"N\":\"1693147200\"}}"
```

Result: All activities for U001 where Timestamp starts with 2025-08-27.

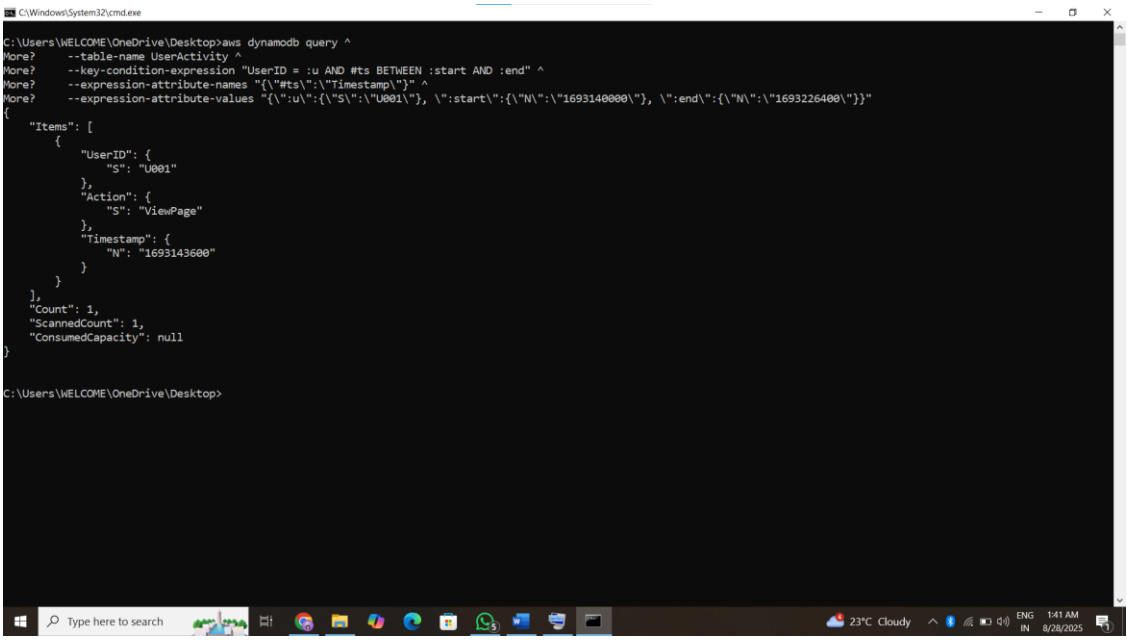
This is great for **prefix queries** (like all logs from a date, or all orders from a given month).

---

3. Query with between on Sort Key

If Timestamp is numeric (e.g., Unix epoch), you can query a date range:

```
aws dynamodb query ^
  --table-name UserActivity ^
  --key-condition-expression "UserID = :u AND #ts BETWEEN :start AND :end" ^
  --expression-attribute-names "{\"#ts\":\"Timestamp\"}" ^
  --expression-attribute-values "{\":u\":{\"S\":\"U001\"}, \":start\":{\"N\":\"1693140000\"},
\":end\":{\"N\":\"1693226400\"}}"
```



Result: All activities for U001 between **1693140000** and **1693226400** (e.g., between two timestamps).

---

4. Learning Outcomes

- query always **needs a partition key**.
- Sort key can be filtered using operators:
  - =, <, <=, >, >=
  - between
  - begins_with
- Results are always sorted by the sort key (ascending; you can set --scan-index-forward false for descending).